# Creating an LLVM Runtime

## EuroLLVM 2026

Michael Kruse

Advanced Micro Devices GmbH

14th April, 2026

# Outline

Introduction

Let's Create a Runtime

And They Lived Happily...



AMD
together we advance_

# Outline

**Introduction**
- Why This Tutorial?
- History
- Rationale
- Goal

**Let's Create a Runtime**

**And They Lived Happily...**



AMD
together we advance_

# Why This Tutorial

[redacted] 1 hour ago Member

After this change, it's no longer possible to run openmp tests in a non-bootstrapping build (i.e. without building clang at the same time).

Previously it was possible to run the tests in a standalone build, by building the `FileCheck` and `not` binaries and pointing at them with `OPENMP_LLVM_TOOLS_DIR` - like I did at mstorsjo/llvm-mingw@ `20260210` /.github/workflows/build.yml#L583-L612.

Now after this change (or before, when doing a runtimes build), running tests only works if there's a `FileCheck` target already in the build, i.e. if doing a bootstrapping build:

llvm-project/openmp/cmake/OpenMPTesting.cmake
Lines 52 to 74 in c6425aa

```
52    if (${OPENMP_STANDALONE_BUILD})
53      find_standalone_test_dependencies()
54
55      # Set lit arguments.
56      set(DEFAULT_LIT_ARGS "-sv --show-unsupported --show-xfail")
57      if (MSVC OR XCODE)
58        set(DEFAULT_LIT_ARGS "${DEFAULT_LIT_ARGS} --no-progress-bar")
59      endif()
60      if ("${CMAKE_SYSTEM_NAME}" MATCHES "AIX")
61        set(DEFAULT_LIT_ARGS "${DEFAULT_LIT_ARGS} --time-tests --timeout=3000")
62      endif()
63      set(OPENMP_LIT_ARGS "${DEFAULT_LIT_ARGS}" CACHE STRING "Options for lit.")
```



AMD
together we advance_

# Why This Tutorial

[redacted]

[redacted] on Dec 8, 2025 • edited

Yep, just for context from the Rust side. We have a lot of builders in CI, but the slowest one we have is the x86 dist builder, which we use to e.g. create compiler artifacts. We already build LLVM there around 4 times due to PGO and Bolt, so we really try hard not to make it any slower, since we are already at ~3.5 hrs, and we run this CI before any merge into main.

One of the compromises there is that we have two checkouts of LLVM - once as a standalone llvm/llvm-project build (A) and once as a submodule of rustc in `src/tools/llvm-project`. We use build A to compile C/C++ code, and keep rebuilding the second checkout (B) during bootstrapping.
We only build clang as part of our LLVM Build A. However, rustc is based on LLVM B, and as such we'd also want the openmp and offload libraries to also be part of our LLVM Build B, which does not build the in-tree clang to save compile times.

We've tried for quite a bit to get the runtimes to build despite no in-tree clang or lld in LLVM B, and failed, I'm happy to share all the issues I ran into there. A standalone build instead allows me to use clang from build A to build offload+openmp from checkout B, that's compatible with what we have in rustc. So until that is fixed, standalone builds are likely the only builds that our infra team would be ok with.



AMD
together we advance_

# Why This Tutorial

[redacted] on Dec 8, 2025 • edited

Yep, just for context from the Rust side. We have a lot of builders in CI, but the slowest one we have is the x86 dist builder, which we use to e.g. create compiler artifacts. We already build LLVM there around 4 times due to PGO and Bolt, so we really try hard not to make it any slower, since we are already at ~3.5 hrs, and we run this CI before any merge into main

[redacted] 1 hour ago • edited Contributor

We were using the OpenMP standalone build mode in order to build the static libomp.a runtime library which cannot be build together with the shared libomp.so (there is no FLANG_RT_ENABLE_SHARED and FLANG_RT_ENABLE_STATIC equivalent for libomp, the LIBOMP_ENABLE_SHARED flag is binary: one can get either shared or static library, but never both in the same build).

The OpenMP standalone build mode has been removed by llvm/llvm-project#182022

This complicates our situation. Considering almost nonexistent demand for the static OpenMP runtime library, it is not worth pursuing it.



AMD
together we advance_

# Why This Tutorial

[redacted] on Dec 8, 2025 • edited

Yep, just for context from the Rust side. We have a lot of builders in CI, but the slowest one we have is the x86 dist builder, which we use to e.g. create compiler artifacts. We already build LLVM there around 4 times due to PGO and Bolt, so we really try hard not to make it any slower, since we are already at ~3.5 hrs and ...

[redacted] on Jan 7 • edited

Member

That comes with two drawbacks: 1) checking out `llvm` alone (I'm not talking about `llvm-project`) takes a lot of time. 2) the way to pass CMake arguments to runtime build is extremely user unfriendly. If these two can't be improved, I strongly object the removal of standalone build.

We we
build t
for libc
build).

The Op

This co... pleates our situation. Considering almost nonexistent demand for the static OpenMP runtime library, it is not worth pursuing it.

om



AMD
together we advance_

# Why This Tutorial

[redacted] on Dec 8, 2025 • edited

Yep, just for context from the R

[redacted] on Dec 8, 2025 Member

Rust colleagues rely on standalone builds for their offloading extensions (see rust-lang/rust#148671).

CC @ZuseZ4

the removal of standalone build.

build).

The Op

This co

pursuing it.

time. 2) the

be improved, I strongly object

om

almost nonexistent demand for the static OpenMP runtime library, it is not worth



AMD
together we advance_

# Why This Tutorial

[redacted] on Dec 8, 2025 • edited

Yep, just for context from the [redacted]

[redacted] on Jul 26, 2025

It is definitely not the same!
For standalone build, I used to not needing the llvm-project tree. I could build libomp by just downloading the OpenMP release tarball: `llvmorg-21.1.0-rc1` openmp-21.1.0-rc1.src.tar.xz (download)

I just realized that this workflow got bricked by #62240

Member

Member

Member

time. 2) the

will be improved, I strongly object

[redacted] complicates our situation. Considering almost nonexistent demand for the static OpenMP runtime library, it is not worth pursuing it.



AMD
together we advance_

# Why This Tutorial

[redacted] on Dec 8, 2025 • edited

Yep, just for context from the [redacted]

Member

[redacted] on Jan 7

Member

I don't have a build tree. That's the whole point of building standalone. And the installed binary tree doesn't work.

I just realized that this w[redacted]

[redacted] complicates our situation. Considering almost nonexistent demand for the static Openm[redacted]

pursuing it.



AMD
together we advance_

# History

1. Initially, each subproject had its own SVN repository
   - Subprojects categorized as "tools" (clang: 2009), "runtime" (libprofile), and "projects" (e.g. llvm-gcc)
   - An example project boilerplate existed

[1] https://reviews.llvm.org/D20992
[2] https://lists.llvm.org/pipermail/llvm-dev/2020-October/145997.html
[3] https://reviews.llvm.org/D26365
[4] https://reviews.llvm.org/D132478



AMD
together we advance_

# History

1. Initially, each subproject had its own SVN repository
   - Subprojects categorized as "tools" (clang: 2009), "runtime" (libprofile), and "projects" (e.g. llvm-gcc)
   - An example project boilerplate existed
2. "runtime" subdirectory disappears with dysfunct PGO removal (2013)

[1] https://reviews.llvm.org/D20992
[2] https://lists.llvm.org/pipermail/llvm-dev/2020-October/145997.html
[3] https://reviews.llvm.org/D26365
[4] https://reviews.llvm.org/D132478



AMD
together we advance_

# History

1. Initially, each subproject had its own SVN repository
   - Subprojects categorized as "tools" (clang: 2009), "runtime" (libprofile), and "projects" (e.g. llvm-gcc)
   - An example project boilerplate existed
2. "runtime" subdirectory disappears with dysfunct PGO removal (2013)
3. Desire to separate runtimes[1,2] (2016–2020)
   - Somewhat different from compile-time utils/tools/compilers

[1] https://reviews.llvm.org/D20992
[2] https://lists.llvm.org/pipermail/llvm-dev/2020-October/145997.html
[3] https://reviews.llvm.org/D26365
[4] https://reviews.llvm.org/D132478



AMD
together we advance_

# History

1. Initially, each subproject had its own SVN repository
   - Subprojects categorized as "tools" (clang: 2009), "runtime" (libprofile), and "projects" (e.g. llvm-gcc)
   - An example project boilerplate existed
2. "runtime" subdirectory disappears with dysfunct PGO removal (2013)
3. Desire to separate runtimes[1,2] (2016–2020)
   - Somewhat different from compile-time utils/tools/compilers
4. Introduction of `LLVM_ENABLE_PROJECTS`[3] (2016)
5. Introduction of `LLVM_ENABLE_RUNTIMES`[4] (2022)
   - First for libc++

[1] https://reviews.llvm.org/D20992
[2] https://lists.llvm.org/pipermail/llvm-dev/2020-October/145997.html
[3] https://reviews.llvm.org/D26365
[4] https://reviews.llvm.org/D132478



AMD
together we advance_

# Rationale

## Why LLVM_ENABLE_RUNTIMES?

### Build Host

The system we build on

- cmake, ninja, cc, …
- llvm-tblgen, mlir-tblgen, …

### Cross-Compile Target

Where Clang will run

- clang, opt, lld, …
- LLVMSupport{.a/.so}, LLVMPolly.so, …

### Compiler Target

What Clang will compile for

- a.out
- compiler-rt, libc, libc++, …

- Automate multiple build-steps
  - Instead of: Build clang → build compiler-rt for x86_64 → build compiler-rt for arm64 → …
- Reduce per-runtime boilerplate/differences between build modes
- Runtimes compiled (Stage 2) using just-built Clang



AMD
together we advance_

# Rationale

## Why LLVM_ENABLE_RUNTIMES?

### Build Host

The system we build on

- cmake, ninja, cc, …
- llvm-tblgen, mlir-tblgen, …

### Cross-Compile Target

Where Clang will run

- clang, opt, lld, …
- LLVMSupport{.a/.so}, LLVMPolly.so, …

### Compiler Target

What Clang will compile for

- a.out
- compiler-rt, libc, libc++, …

Stage 1

Stage 2

- Automate multiple build-steps
  - Instead of: Build clang → build compiler-rt for x86_64 → build compiler-rt for arm64 → …
- Reduce per-runtime boilerplate/differences between build modes
- Runtimes compiled (Stage 2) using just-built Clang



AMD
together we advance_

# Tutorial Runtime

## What is our tutorial runtime going to do?

### Our Runtime

```
void _dublin_hello_world() {
  puts("Dia daoibh, a dhomhain!\n");
}
```

- Use LLVM best-practices
- Linux only
- Prioritize conciseness over
  - ... features
  - ... comments
  - ... boilerplate not topic in this tutorial
- Point out shortcomings of the current system



AMD
together we advance_

# Outline

Introduction

**Let's Create a Runtime**
- Setup
- Step 1: Register with the LLVM Build System
- Step 2: Build a Library Artifact
- Step 3: An Example that uses the Library
- Step 4: Build Modes
- Step 5: CMake Cache Files
- Step 6: Artifact Output Location
- Step 7: Installation
- Step 8: Shared and Static Libraries
- Step 9: Regression Tests
- Step 10: Unittests
- Step 11: Sphinx Docs
- Step 12: Doxygen Docs
- Step 13: Cross-Compilation
- Step 14: Accelerator Offloading
- Step 15: Depending on Other LLVM Libraries / C++
- Step ?

And They Lived Happily...



AMD
together we advance_

# Setup

`https://github.com/Meinersbur/llvm-dublin/tree/dublin-0-main`

- Update Readme for tutorial
  - `README.md`
- Make git ignore `install*` subdirs (like `build*` already is)
  - `.gitignore`



AMD
together we advance_

# Register with the LLVM Build System

https://github.com/Meinersbur/llvm-dublin/tree/dublin-1-basic

- Allow `LLVM_ENABLE_RUNTIME=dublin`
  - `llvm/CMakeLists.txt`
  - `runtimes/CMakeLists.txt`
- Subproject boilerplate
  - `dublin/CMakeLists.txt`
  - `dublin/CODE_OWNERS.TXT`
  - `dublin/LICENSE.TXT`



AMD
together we advance_

# Step 2: Build a Library Artifact

- Library source files
  - `lib/Dublin/dublin.c`
  - `include/dublin/Dublin/dublin.h`
- Build `libdublin.a`
  - `add_executable(dublin dublin.c)`



AMD
together we advance_

# Standard Subproject Layout

```
subproject
├─ docs
│  └─ *.rst, *.md
├─ examples
│  └─ Example1
│     └─ *.c, *.cpp
├─ test
│  ├─ Unit
│  │  └─ lit.cfg.py, lit.site.cfg.py.in
│  ├─ Component1
│  │  └─ *
│  ├─ Component2
│  │  └─ *
│  └─ lit.cfg.py, lit.site.cfg.py.in
├─ unittests
│  ├─ Component1
│  │  └─ Component1Test.cpp
│  └─ Component2
│     └─ Component1Test.cpp
├─ cmake
│  ├─ Modules/modules
│  │  └─ *.cmake
│  └─ caches
│     └─ *.cmake
├─ include
│  └─ subproject
│     ├─ Component1
│     │  └─ component1.h
│     └─ Component2
│        └─ component2.h
├─ lib
│  ├─ Component1
│  │  ├─ component1.c
│  │  └─ component1-internal.h
│  └─ Component2
│     └─ component2.c
└─ utils
   └─ utility
      └─ utility.cpp
```



AMD
together we advance_

# An Example that uses the Library

https://github.com/Meinersbur/llvm-dublin/tree/dublin-3-example

- Example source
  - examples/dublin-hello/dublin-hello.c
- Build instructions
  - examples/dublin-hello/CMakeLists.txt



AMD
together we advance_

# Build Modes

- Detecting Legacy Build Modes
  - `CMakeLists.txt`



AMD
together we advance_

# Build Modes

## Legacy Build Modes

**As project:** `cmake -S <llvm-project>/llvm -D LLVM_ENABLE_PROJECTS=dublin`
**Standalone:** `cmake -S <llvm-project>/dublin`

- Intended to be superseded by runtimes build modes[1]
- If your build infrastructure still uses this, please consider updating

## Runtimes Build Modes

**Bootstrapping:** `cmake -S <llvm-project>/llvm -D LLVM_ENABLE_RUNTIMES=dublin`
**Default/Standalone:** `cmake -S <llvm-project>/runtimes -D LLVM_ENABLE_RUNTIMES=dublin`

- Shared+common boilerplate code in `<llvm-project>/runtimes/CMakeLists.txt`

[1] https://lists.llvm.org/pipermail/llvm-dev/2020-October/145997.html



AMD
together we advance_

# Bootstrapping Runtimes Build Mode

▶



AMD
together we advance_

# Default/Standalone Runtimes Build Mode

▶



AMD
together we advance_

# CMake Cache Files

▶



AMD
together we advance_

# Artifact Output Location

▶



AMD
together we advance_

# Installation

▶



AMD
together we advance_

# Shared and Static Libraries

- Add a Configuration option
-



AMD
together we advance_

# Regression Tests

▶



AMD
together we advance_

# Unittests

▶



AMD
together we advance_

# Sphinx Docs

▶



AMD
together we advance_

# Doxygen Docs

▶



AMD
together we advance_

# Cross-Compilation

▶



AMD
together we advance_

# Accelerator Offloading

▶



AMD
together we advance_

# Depending on Other LLVM Libraries / C++

▶



AMD
together we advance_

# Platform Introspection

▶



AMD
together we advance_

# Outline

Introduction

Let's Create a Runtime

**And They Lived Happily...**
- More Topics
- Build System Improvements
- Conclusion



AMD
together we advance_

# Uncovered Topics

- Platform Support: Windows/MacOS/AIX/Fuchsia/...
- Multilib
- ...



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target
- Fat libraries



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target
- Fat libraries
- Get rid of `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR`



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target
- Fat libraries
- Get rid of `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR`
- QEMU (`CMAKE_CROSSCOMPILING_EMULATOR`)



AMD
together we advance_

# Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target
- Fat libraries
- Get rid of `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR`
- QEMU (`CMAKE_CROSSCOMPILING_EMULATOR`)
- ...



AMD
together we advance_

# Conclusion

- Now you know how `LLVM_ENABLE_RUNTIMES` works
- Use the Knowledge Wisely
- I currently cannot recommend working on improving the system, waste of your time



AMD
together we advance_

AMD